HASKISH - POSSIBLE SOLUTIONS TO LESSONS

This file contains possible solutions to the challenges in lessons.txt.
Remember: There are often multiple valid solutions to each problem.

==========
Lesson 1 - Using the REPL
==========

No Challenges

==========
Lesson 2 - Variables
==========

No Challenges

==========
Lesson 3 - Simple Functions
==========

Question
tripleMe - Accepts one parameter and returns three times its value

Solution
```
tripleMe x = x * 3
```

Question
doubleAdd - Accepts two parameters, doubles each of them, then adds them together

Solution
```
doubleAdd x y = x * 2 + y * 2
```

Question
subtractThenDouble - Accepts two parameters, subtracts the second from the first, then doubles the result

Solution
```
subtractThenDouble x y = (x - y) * 2
```

Question
isGreaterThan100 - Accepts two parameters, adds them together, and returns True if the sum is greater than 100

Solution
```
isGreaterThan100 x y = x + y > 100
```

Question
averageTwo - Accepts two parameters and returns their average

Solution
```
averageTwo x y = (x + y) / 2
```

Question
minutesToHours - Accepts a number of minutes and returns how many complete hours

Solution
```
minutesToHours m = div m 60
```

Question
isEven - Accepts one parameter and returns True if the number is even

Solution
```
isEven n = mod n 2 == 0
```

==========
Lesson 4 - Partial Function Application
==========

Question
Create subtract x y = x - y, then use partial application to create subtractFromHundred

Solution
```
subtract x y = x - y
subtractFromHundred = subtract 100
```

Question
Create addSuffix suffix text = text ++ suffix, then use partial application to create addExclamation

Solution
```
addSuffix suffix text = text ++ suffix
addExclamation = addSuffix "!"
```

Question
Use the built-in min function with partial application to create capAt100

Solution
```
capAt100 = min 100
```

Question
Create addPrefix prefix text = prefix ++ text, then use partial application to create addMr

Solution
```
addPrefix prefix text = prefix ++ text
addMr = addPrefix "Mr. "
```

==========
Lesson 5 - First-Class Functions
==========

Question
Write a function applyThrice that takes a function and a value, and applies the function three times

Solution
```
applyThrice f x = f (f (f x))
```

Question
Write a function applyToFive that takes a function as an argument and applies it to the number 5

Solution
```
applyToFive f = f 5
```

Question
Write a function doTwice that takes a function f and a value x, and returns a function that when called with a value y, computes f (f x) + y

Solution
```
doTwice f x = \y -> f (f x) + y
```

Question

Write a function applyBoth that takes two functions f and g, and a value x, then returns f x + g x

Solution
```
applyBoth f g x = f x + g x
```

==========
Lesson 6 - Function Composition
==========

Question
Define addFive x = x + 5 and double x = x * 2, then create addFiveThenDouble using composition

Solution
```
addFive x = x + 5
double x = x * 2
addFiveThenDouble = double . addFive
```

Question
Define square x = x * x and addThree x = x + 3, then create squareThenAddThree using composition

Solution
```
square x = x * x
addThree x = x + 3
squareThenAddThree = addThree . square
```

Question
Create a composition addThenDoubleSquared that adds 1, doubles the result, then squares it

Solution
```
addOne x = x + 1
double x = x * 2
square x = x * x
addThenDoubleSquared = square . double . addOne
```

Question
Define half x = x / 2 and square x = x * x. Create two different compositions: halfThenSquare and squareThenHalf

Solution
```
half x = x / 2
square x = x * x
halfThenSquare = square . half
squareThenHalf = half . square
```

Question
Create a function secondElement that gets the second element of a list using composition

Solution
```
secondElement = head . tail
```

==========
Lesson 7 - Using the Function Panel
==========

Question
isRightTriangle - Accepts three parameters (sides of a triangle) and returns True if they form a right-angled triangle

Solution
```
square x = x * x
sumOfSquares a b = square a + square b
isRightTriangle a b c = sumOfSquares a b == square c
```

```
================================================================================
Lesson 8 - Building More Complex Programs
================================================================================

Question
Write a program to analyse quadratic equations (ax² + bx + c = 0)
- discriminant a b c - Calculates b² - 4ac
- hasRealRoots a b c - Returns True if the equation has at least one real root
- hasTwoRoots a b c - Returns True if the equation has two distinct real roots
- hasOneRoot a b c - Returns True if the equation has exactly one root

Solution
square x = x * x
discriminant a b c = square b - 4 * a * c
hasRealRoots a b c = discriminant a b c >= 0
hasTwoRoots a b c = discriminant a b c > 0
hasOneRoot a b c = discriminant a b c == 0


================================================================================
Lesson 9 - Introduction to Lists
================================================================================

No Challenges (Questions only)


================================================================================
Lesson 10 - Pattern Matching & Recursion
================================================================================

Question
fib - Calculates the nth Fibonacci number

Solution
fib 0 = 0
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)

Note: This solution works but becomes extremely slow for larger values like fib 35.
Why? Because it recalculates the same values many, many times!

For example, calculating fib 5:
- fib 5 calls fib 4 and fib 3
- fib 4 calls fib 3 and fib 2
- fib 3 is calculated twice already!
- This exponential explosion means fib 35 requires billions of function calls

The Recursion examples shows a much faster version called "fastFib" that uses tail
recursion with accumulators - it calculates fib 1000 almost instantly by avoiding
redundant calculations. This demonstrates that choosing the right algorithm matters!


Question
sumList - adds all numbers in a list

Solution
sumList [] = 0
sumList (x:xs) = x + sumList xs


Question
productList - multiplies all numbers in a list

Solution
productList [] = 1
productList (x:xs) = x * productList xs
```

Question
replicateEach - replicates each element in a list twice

Solution
```
replicateEach [] = []
replicateEach (x:xs) = x : x : replicateEach xs
```

Question
appendToEach - appends a given element to the end of each sublist

Solution
```
appendToEach e [] = []
appendToEach e (x:xs) = (x ++ [e]) : appendToEach e xs
```

Question
flatten - flattens a list of lists into a single list

Solution
```
flatten [] = []
flatten (x:xs) = x ++ flatten xs
```

Question
pairUp - pairs up consecutive elements from a list

Solution
```
pairUp [] = []
pairUp [x] = []
pairUp (x:y:xs) = (x, y) : pairUp xs
```

Question
countOccurrences - counts how many times an element appears in a list

Solution
```
boolToInt True  = 1
boolToInt False = 0

countOccurrences e [] = 0
countOccurrences e (x:xs) = boolToInt (e == x) + countOccurrences e xs
```

==========
Lesson 11 - Guards & Conditionals
==========

Question
switchBinary - Given a list of binary digits, return a new list with 0s changed to 1s and 1s changed to 0s

Solution
```
switchBinary [] = []
switchBinary (x:xs)
  | x == 0    = 1 : switchBinary xs
  | x == 1    = 0 : switchBinary xs
  | otherwise = switchBinary xs
```

Question
countTwoElems - Count how many times either of two given elements appear in a list

Solution
```
countTwoElems e1 e2 [] = 0
countTwoElems e1 e2 (x:xs)
  | x == e1   = 1 + countTwoElems e1 e2 xs
  | x == e2   = 1 + countTwoElems e1 e2 xs
  | otherwise = countTwoElems e1 e2 xs
```

Question
inList - Return True if a given element appears in the list, False otherwise

Solution
```
inList e [] = False
inList e (x:xs)
  | e == x    = True
  | otherwise = inList e xs
```

Question
removeAll - Remove all occurrences of a given element from a list

Solution
```
removeAll e [] = []
removeAll e (x:xs)
  | e == x    = removeAll e xs
  | otherwise = x : removeAll e xs
```

Question
removeFirst - Remove only the first occurrence of a given element from a list

Solution
```
removeFirst e [] = []
removeFirst e (x:xs)
  | e == x    = xs
  | otherwise = x : removeFirst e xs
```

Question
replace - Replace all instances of one element with another element

Solution
```
replace old new [] = []
replace old new (x:xs)
  | x == old  = new : replace old new xs
  | otherwise = x : replace old new xs
```

Question
multEvenIdx - Multiply all values at even-numbered indices (0, 2, 4, ...)

Solution
```
multEvenIdxHelper n [] = 1
multEvenIdxHelper n (x:xs)
  | mod n 2 == 0 = x * multEvenIdxHelper (n + 1) xs
  | otherwise    = multEvenIdxHelper (n + 1) xs

multEvenIdx xs = multEvenIdxHelper 0 xs
```

Question
multEvenNums - Multiply all the even numbers in a list

Solution
```
multEvenNums [] = 1
multEvenNums (x:xs)
  | mod x 2 == 0 = x * multEvenNums xs
  | otherwise    = multEvenNums xs
```

Question
insertEvenPos - Insert a given element at all even positions in a list

Solution
```
insertEvenPosHelper e n [] = [e]
```

```
insertEvenPosHelper e n (x:xs)
  | mod n 2 == 0 = e : x : insertEvenPosHelper e (n + 1) xs
  | otherwise    = x : insertEvenPosHelper e (n + 1) xs

insertEvenPos e xs = insertEvenPosHelper e 0 xs
```

Question
multBy - Multiply every element in a list by a given number

Solution
```
multBy n [] = []
multBy n (x:xs) = (n * x) : multBy n xs
```

==========
Lesson 12 - Higher-Order Functions: Map, Filter, and Fold
==========

Question
Use map to convert a list of temperatures from Fahrenheit to Celsius

Solution
```
fahrenheitToCelsius f = (f - 32) * 5 / 9
map fahrenheitToCelsius [32,50,68,86,104]
```

Question
Use map to add an exclamation mark to the end of each string in a list

Solution
```
map (++ "!") ["Hello","Goodbye","Welcome"]
```

Question
Use map to get the first character of each string in a list

Solution
```
map head ["apple","banana","cherry"]
```

Question
Use filter to keep only numbers greater than 10

Solution
```
filter (>10) [5,15,3,20,8,25]
```

Question
Use filter to keep only strings that start with the letter 'a'

Solution
```
startsWithA s = head s == 'a'
filter startsWithA ["apple","banana","apricot","cherry"]
```

Question
Use filter to keep only the odd numbers from a list

Solution
```
isOdd x = mod x 2 /= 0
filter isOdd [1,2,3,4,5,6,7,8,9,10]
```

Question
Use fold to find the sum of all even numbers from 1 to 10

Solution
```
isEven x = mod x 2 == 0
```

```
fold (+) 0 (filter isEven [1,2,3,4,5,6,7,8,9,10])
```

Question
Use fold to concatenate a list of strings

Solution
```
fold (++) "" ["Hello"," ","World","!"]
```

Question
Use fold to count how many elements are in a list

Solution
```
countElem acc x = acc + 1
fold countElem 0 [10,20,30,40,50]
```

Question
Get the sum of squares of all even numbers in a list

Solution
```
square x = x * x
isEven x = mod x 2 == 0
fold (+) 0 (map square (filter isEven [1,2,3,4,5,6]))
```

Question
Double all numbers greater than 5, then find their product

Solution
```
fold (*) 1 (map (*2) (filter (>5) [3,6,2,8,4,10]))
```

Question
Extract the length of each string, then sum the lengths

Solution
```
fold (+) 0 (map length ["hello","world","functional"])
```

==========
Lesson 13 - List Comprehensions
==========

Question
Create a list of the first 10 perfect squares

Solution
```
[x * x | x <- [1..10]]
```

Question
Create a list that adds 5 to every number from 1 to 8

Solution
```
[x + 5 | x <- [1..8]]
```

Question
Create a list of all numbers from 1 to 20 that are divisible by 3

Solution
```
[x | x <- [1..20], mod x 3 == 0]
```

Question
Create a list of all even numbers from 1 to 20, then square each one

Solution
```
[x * x | x <- [1..20], mod x 2 == 0]
```

Question
Create a list of all numbers from 1 to 50 that are divisible by both 3 and 5

Solution
```
[x | x <- [1..50], mod x 3 == 0, mod x 5 == 0]
```

Question
Extract all lowercase vowels from the string "Functional Programming"

Solution
```
isVowel c = c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u'
[c | c <- "Functional Programming", isVowel c]
```

Question
Create a multiplication table: all products of x and y where both x and y range from 1 to 5

Solution
```
[x * y | x <- [1..5], y <- [1..5]]
```

Question
Create all possible pairs of a number from [1,2,3] and a letter from "abc"

Solution
```
[(x, y) | x <- [1,2,3], y <- "abc"]
```

Question
Generate all Pythagorean triples (a,b,c) where a² + b² = c² and all values are between 1 and 20

Solution
```
[(a,b,c) | a <- [1..20], b <- [1..20], c <- [1..20], a*a + b*b == c*c]
```

Question
Generate the first 10 triangular numbers

Solution
```
sum xs = fold (+) 0 xs
[sum [1..n] | n <- [1..10]]
```

Question
Flatten a list of lists, but only including sublists that have more than 2 elements

Solution
```
[x | xs <- [[1],[2,3],[4,5,6],[7,8,9,10]], length xs > 2, x <- xs]
```

Question
Find all perfect numbers from 1 to 100

Solution
```
divisors n = [x | x <- [1..n-1], mod n x == 0]
sum xs = fold (+) 0 xs
isPerfect n = sum (divisors n) == n
[n | n <- [1..100], isPerfect n]
```

==========
Lesson 14 - Tuples and Destructuring
==========

Question
fst - returns the first element of a pair

Solution
```
fst (x, y) = x
```

Question
snd - returns the second element of a pair

Solution
```
snd (x, y) = y
```

Question
swap - swaps the elements of a pair

Solution
```
swap (x, y) = (y, x)
```

Question
both - applies a function to both elements of a pair

Solution
```
both f (x, y) = (f x, f y)
```

Question
first - applies a function to only the first element of a pair

Solution
```
first f (x, y) = (f x, y)
```

Question
second - applies a function to only the second element of a pair

Solution
```
second f (x, y) = (x, f y)
```

Question
getFirst - returns the first element of a triple

Solution
```
getFirst (a, b, c) = a
```

Question
getThird - returns the third element of a triple

Solution
```
getThird (a, b, c) = c
```

Question
rotateTriple - rotates elements: (a,b,c) becomes (b,c,a)

Solution
```
rotateTriple (a, b, c) = (b, c, a)
```

Question
sumPairList - sums the elements in each pair and returns a list of sums

Solution
```
sumPairList [] = []
sumPairList ((x,y):xs) = (x + y) : sumPairList xs
```

Question
filterByFirst - keeps only pairs where the first element is greater than a given value

Solution
```
filterByFirst n [] = []
filterByFirst n ((x,y):xs)
  | x > n     = (x,y) : filterByFirst n xs
  | otherwise = filterByFirst n xs
```

Question
maxPairs - returns the maximum of each pair

Solution
```
maxPairs [] = []
maxPairs ((x,y):xs) = max x y : maxPairs xs
```

Question
zip - takes two lists and returns a list of pairs, combining corresponding elements

Solution
```
zip [] ys = []
zip xs [] = []
zip (x:xs) (y:ys) = (x, y) : zip xs ys
```

Question
unzip - separates a list of pairs into two lists

Solution
```
unzip pairs = ([x | (x, y) <- pairs], [y | (x, y) <- pairs])
```

Question
dotProduct - computes the dot product of two vectors

Solution
```
sum xs = fold (+) 0 xs
dotProduct xs ys = sum [x * y | (x, y) <- zip xs ys]
```

Question
zipWith - takes a function and two lists, and combines them element-wise using that function

Solution
```
zipWith f xs ys = [f x y | (x, y) <- zip xs ys]
```

Question
enumerate - pairs each element with its index (starting from 0)

Solution
```
enumerateHelper n [] = []
enumerateHelper n (x:xs) = (n, x) : enumerateHelper (n + 1) xs

enumerate xs = enumerateHelper 0 xs
```

Question
splitAt - takes a number n and a list, and returns a pair of lists: the first n elements and the remaining elements

Solution
```
splitAt n xs = (take n xs, drop n xs)
```

Question
pairwiseSum - takes two lists and returns a list of their pairwise sums

Solution
```
pairwiseSum [] ys = []
pairwiseSum xs [] = []
pairwiseSum (x:xs) (y:ys) = (x + y) : pairwiseSum xs ys
```

Question
mergeAlternate - merges two lists by alternating elements

Solution
```
mergeAlternate [] ys = ys
mergeAlternate xs [] = xs
mergeAlternate (x:xs) (y:ys) = x : y : mergeAlternate xs ys
```

==========
Lesson 15 - Lambda Functions
==========

Question
Use a lambda with map to convert a list of numbers to their negatives

Solution
```
map (\x -> -x) [1,2,3,4,5]
```

Question
Use a lambda with filter to keep only strings longer than 3 characters

Solution
```
filter (\s -> length s > 3) ["hi", "hello", "bye", "world"]
```

Question
Use a lambda with map to extract the second element from each pair

Solution
```
map (\(x,y) -> y) [(1,2), (3,4), (5,6)]
```

Question
Use a lambda with fold to find the sum of all numbers in a list

Solution
```
fold (\acc x -> acc + x) 0 [1,2,3,4]
```

Question
Use map with a lambda to square only the even numbers, leave odd numbers unchanged

Solution
```
map (\x -> if mod x 2 == 0 then x * x else x) [1,2,3,4,5,6]
```

Question
Use filter with a lambda to keep only pairs where both elements are even

Solution
```
filter (\(x,y) -> mod x 2 == 0 && mod y 2 == 0) [(1,2), (4,6), (3,5), (8,10)]
```

Question
Write a function applyThrice that takes a function and a value, and applies the function three times

Solution

```
applyThrice f x = f (f (f x))
```

Question
Use map with a lambda to turn each pair into a sum

Solution
```
map (\(x,y) -> x + y) [(1,2), (3,4), (5,6)]
```

Question
Use fold with a lambda to count how many numbers in a list are greater than 5

Solution
```
fold (\acc x -> if x > 5 then acc + 1 else acc) 0 [1,7,3,9,2,8]
```

Question
Write filterMap that filters a list with one lambda and then maps another lambda over the result

Solution
```
filterMap predicate transform xs = map transform (filter predicate xs)
```

Question
Use map with a lambda to apply different functions to odd and even numbers: multiply odd by 3, even by 2

Solution
```
map (\x -> if mod x 2 == 0 then x * 2 else x * 3) [1,2,3,4,5]
```

Question
Write compose that takes two functions and returns a lambda that applies them in sequence

Solution
```
compose f g = \x -> f (g x)
```

Question
Use fold with a lambda to find the maximum element in a list

Solution
```
fold (\acc x -> if x > acc then x else acc) (head [3,7,2,9,1]) (tail [3,7,2,9,1])
```

Question
Write sumOfSquaresOfEvens using only lambdas with map, filter, and fold

Solution
```
sumOfSquaresOfEvens xs = fold (\acc x -> acc + x) 0 (map (\x -> x * x) (filter (\x -> mod x 2 == 0) xs))
```

Question
Use lambdas to extract all second elements from a list of pairs, double them, keep only those greater than 10, and sum them

Solution
```
processData xs = fold (\acc x -> acc + x) 0 (filter (\x -> x > 10) (map (\x -> x * 2) (map (\(a,b) -> b) xs)))
```

Question
Create a function that takes a list of numbers and returns a list of strings, where even numbers become "EVEN" and odd numbers become "ODD"

Solution
```
map (\x -> if mod x 2 == 0 then "EVEN" else "ODD") [1,2,3,4,5]
```

==
END OF SOLUTIONS
==